National University of Computer & Emerging Sciences, Karachi FAST
School of Computing
Spring 2026, Lab Manual – 09

Course Code: CL-2005	**Course: Database Systems Lab**
Instructor(s):	Mr. Abdullah Shaikh, Mr. Jahanzaib, Mr. Muhammad Khalid, Mr. Osama Tahir, Mr. Sheeraz Iqbal, Ms. Hijaab Sikander, Ms. Kinza Mushtaq, Ms. Ramsha Jat, Ms. Sadaf Zehra, Ms. Saira Ayub, Ms. Ramsha Iqbal

Relational Modeling

SQL Data Modeler
Oracle SQL Developer Data Modelers a standalone, independent product, available for download from the Oracle Technology Network (OTN).SQL Developer Data Modeler runs on Windows, Linux and MacOS X.

Logical Models
The logical model in SQL Developer Data Modeler includes standard logical modeling facilities, such as drawing entities and relationships etc.

Relational Models
The SQL Developer Data Modeler relational model is an intermediate model between the logical model and the physical models. It supports relational design decisions independent of the constraints of the target physical platform(s). All many-to-many relationships and all super type/sub-types entity hierarchies are resolved during forward engineering (transformation) of the logical model, or part of it, to a relational model.
Physical Model
A physical data model defines all of the logical database components and services that are required to build a database or can be the layout of an existing database.
A physical data model consists of the table's structure, column names and values, foreign and primary keys and the relationships among the tables.

Data Modeling for a Small Database
We will use SQL Developer Data Modeler to create models for a simplified library database Entities Included will be-
• Books
• Patrons (people who have library cards)
• Transactions (checking a book out, returning a book, and so on).

We are using only a subset of the possible steps for the Top-Down Modeling approach.
1. Develop the Logical Model.
2. Develop the Relational Model.
3. Generate DDL.
4. Save the Design.

1. Develop the logical model
The logical model for the data base includes three entities
Books (describes each book in the library),
Patrons (describes each person who has a library card), and
Transactions (describes each transaction involving a patron and a book).

Before we create the entities, we will create some domains that will make the entity creation (and later DDL generation) more meaningful and specific.

Adding Domains
In planning for our data needs, we have determined that several kinds of fields will occur in multiple kinds of records, and many fields can share a definition. For example, we have decided that-

- The first and last names of persons can be up to25 characters each.
- Street address lines can be up to 40 characters.
- City names can be up to25 characters.
- State codes (United States) are2-character standard abbreviations.
- Zip codes (United States postal codes) can be up to 10 characters (nnnnn-nnnn).
- Book identifiers can be up to 20 characters.
- Other identifiers are numeric, with up to 7 digits (no decimal places). Titles (books, articles, and so on) can be up to 50 characters.
- These added domains will also be available after we exit Data Modeler and restart it later.

Steps to add domains

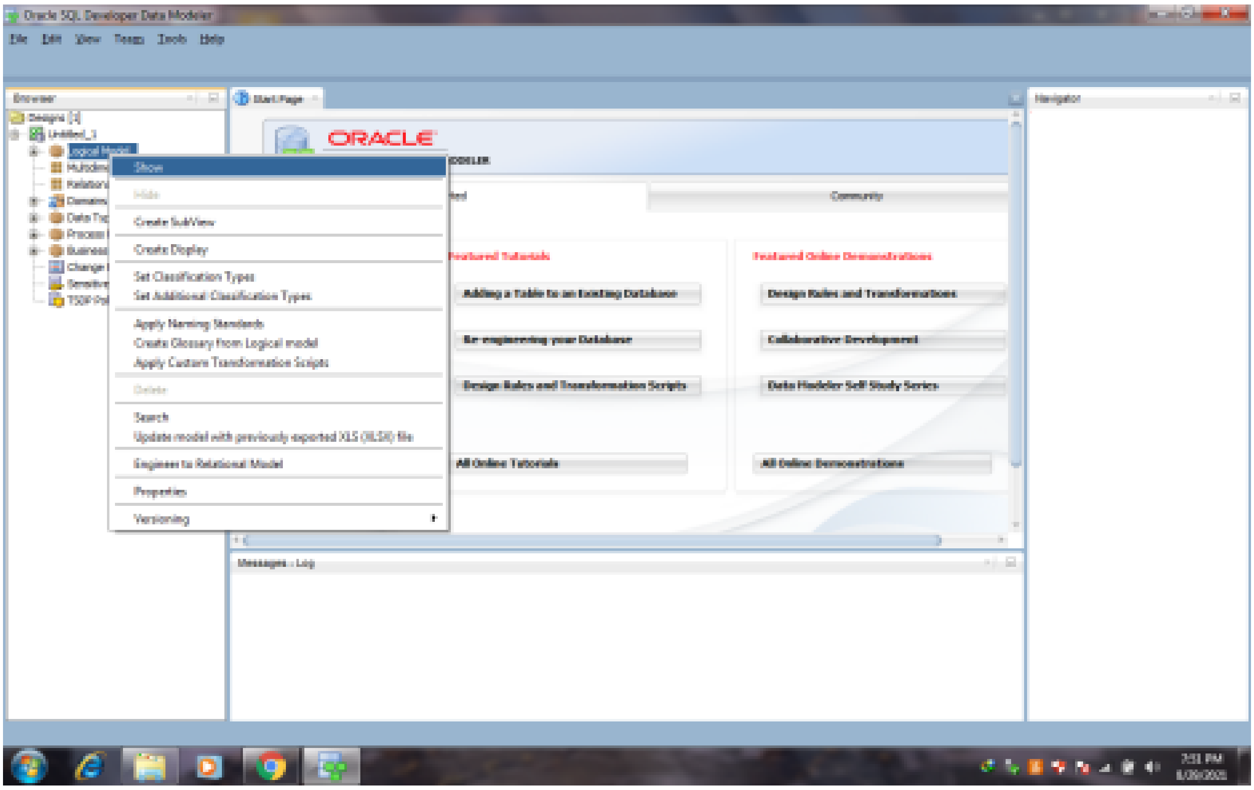
1. Click Tools, then Domains Administration.
2. In the Domains Administration dialog box, add domains with the following definitions.
Click Add to start each definition, and click Apply after each definition.
3. When we have finished defining these domains, click Save. This creates a file named default domains.xml in the domains directory (folder) under the location where we installed Data Modeler.
4. Optionally, copy the default domains.xml file to a new location (not under the Data Modeler installation directory), and give it an appropriate name, such as library_domains.xml. We can then import domains from that file when we create other designs.
5. Click Close to close the dialog box.

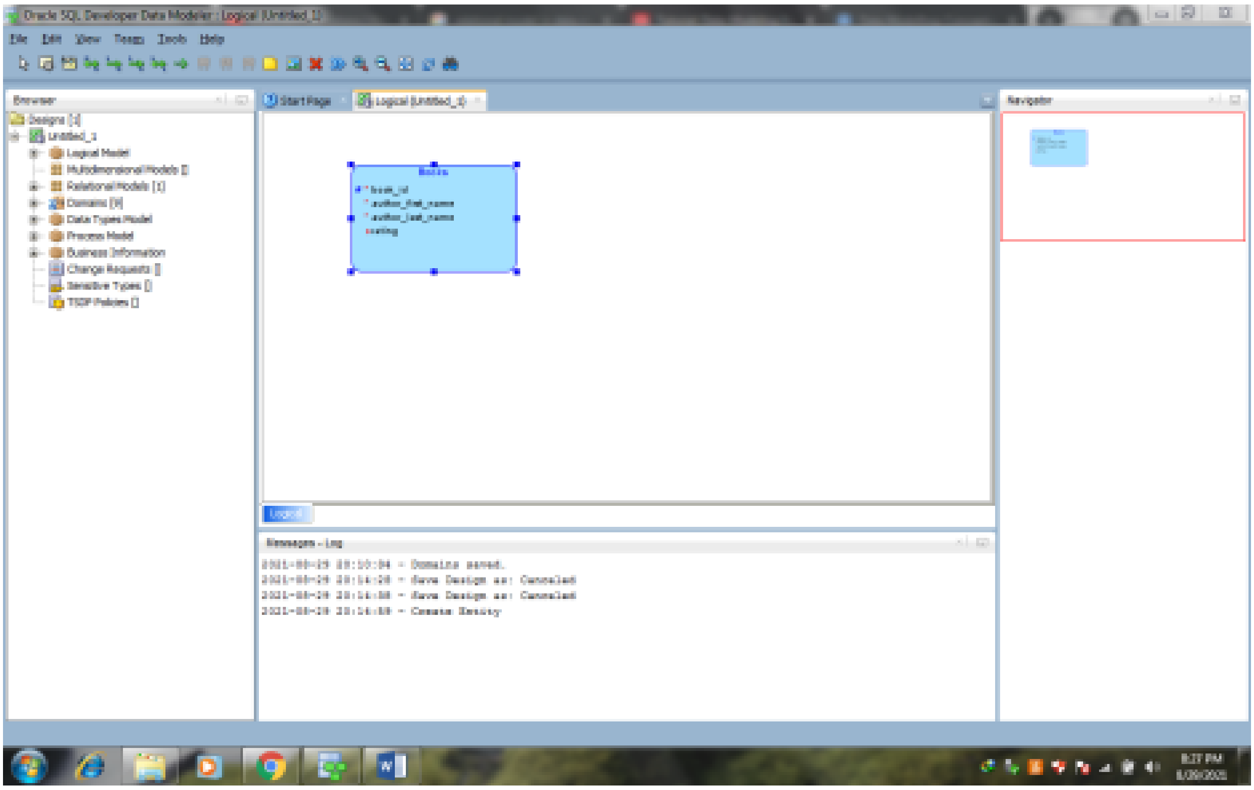
Creating Entities

Creating the book Entity-

- Create the Books entity as follows-
- In the main are a (right side) of the SQL Developer Data Modeler window, click the Logical tab.
- Click the new Entity icon.
- Click in the logical model pane in the main area; and in the Logical pane press,
Diagonally drag, and release the mouse button to draw an entity box. The Entity Properties dialog box is displayed. • Click General on the left, and specify as follows-
- Name- Books
- Click Attributes on the left, and use the Add (+) icon to add the following attributes, one at a time. (For data types, select from the Domain types except or Rating, which is a Logical type.)

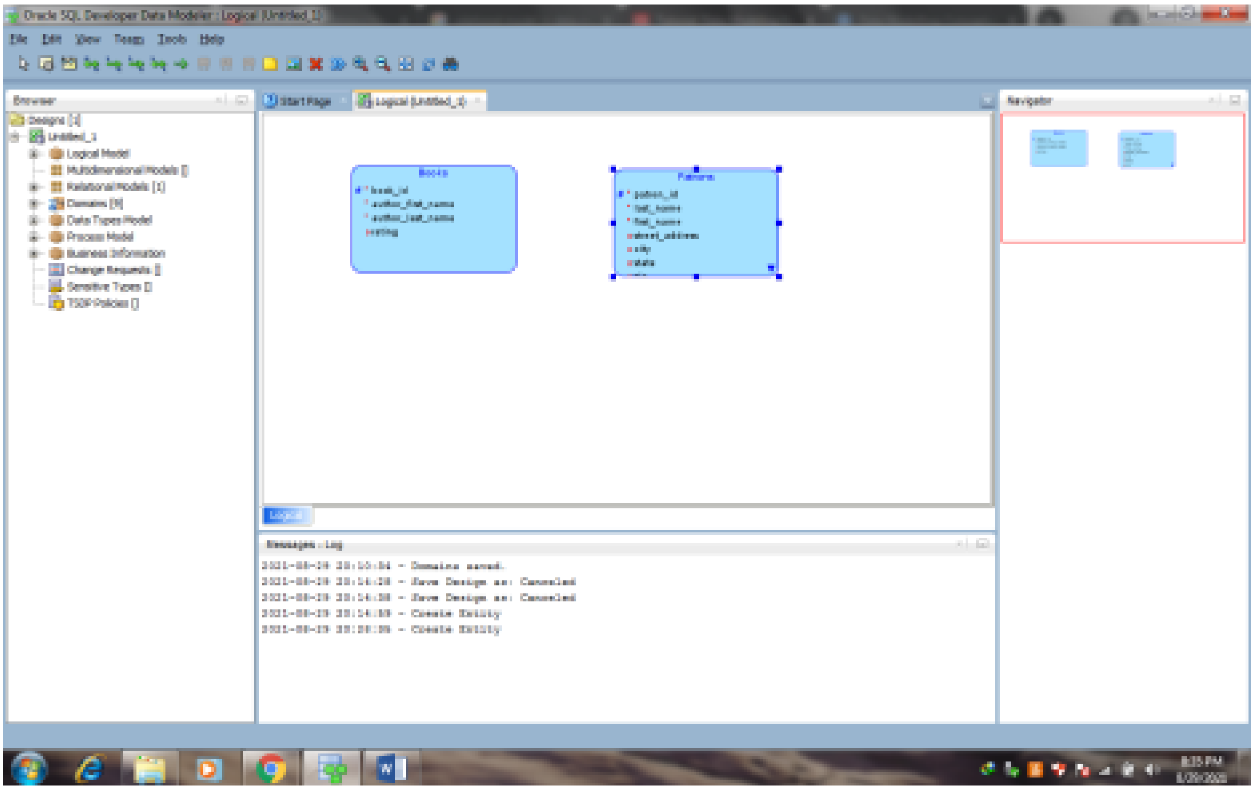
Name	Datatype	Other Information and Notes
book_id	Domain: Book Id	Primary UID (unique identifier). (The Dewey code or other book identifier.)
title	Domain: Title	M (mandatory, that is, must not be null).
author_last_name	Domain: Person Name	M (mandatory, that is, must not be null).
author_first_name	Domain: Person Name	25 characters maximum.
rating	Logical type: NUMERIC (Precision=2, Scale= 0)	(Librarian's personal rating of the book, from 1 (poor) to 10 (great).)





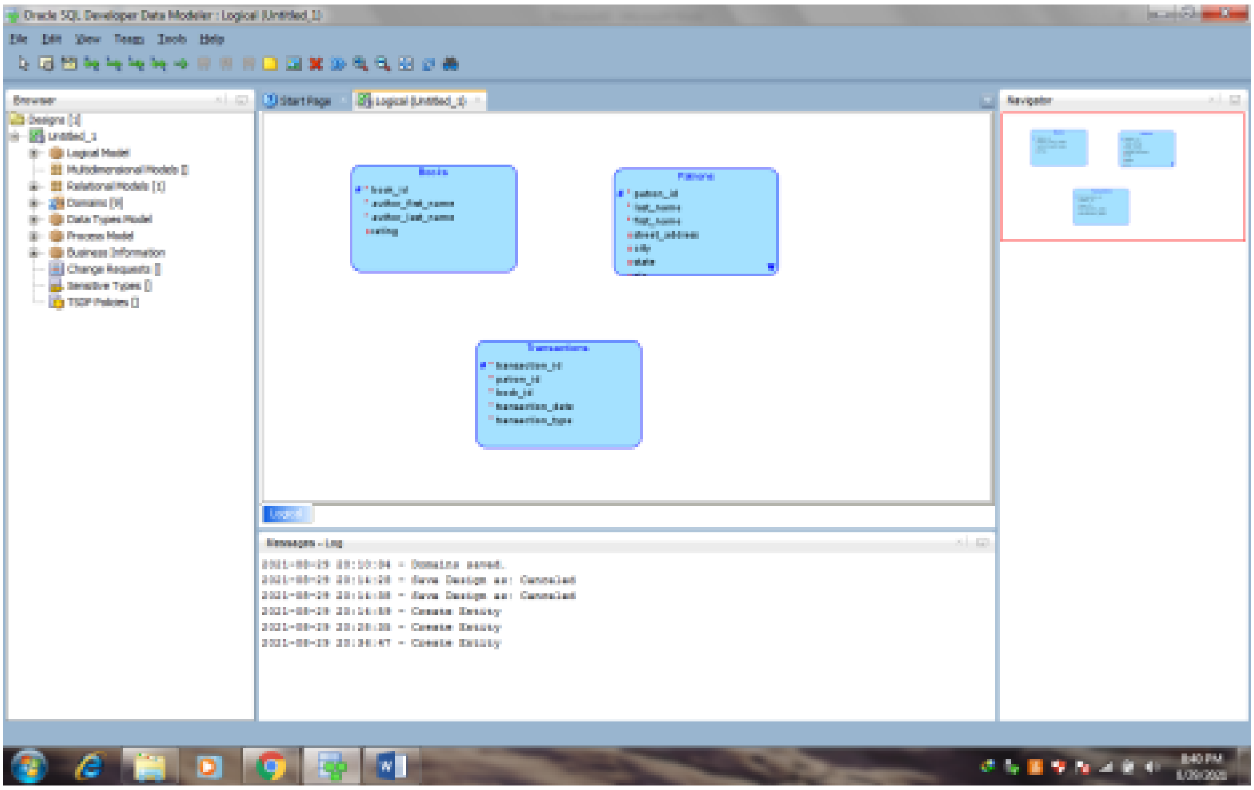
Creating the Patrons Entity

Attribute Name	Type	Other Information and Notes
patron_id	Domain: Numeric Id	Primary UID (unique identifier). (Unique patron ID number, also called the library card number.)
last_name	Domain: Person Name	M (mandatory, that is, must not be null). 25 characters maximum.
first_name	Domain: Person Name	(Patron's first name.)
street_address	Domain: Address Line	(Patron's street address.)
city	Domain: City	(City or town where the patron lives.)
state	Domain: State	(2-letter code for the state where the patron lives.)
zip	Domain: Zip	(Postal code where the patron lives.)
location	Domain:Address	Oracle Spatial geometry object representing the patron's geocoded address.



Creating the Transactions Entity

Attribute Name	Type	Other Information and Notes
tranction_id	Domain: Numeric Id	Primary UID (unique identifier). (Unique transaction ID number)
transaction_date	Logical type: Datetime	M (mandatory, that is, must not be null). Date and time of the transaction.
transaction_type	Domain: Numeric Id	M (mandatory, that is, must not be null). (Numeric code indicating the type of transaction, such as 1 for checking out a book.)



Creating Relations between Entities

Relations show the relationships between entities- one-to-many, many-to-one, or many-to-many. The following relationships exist between the entities

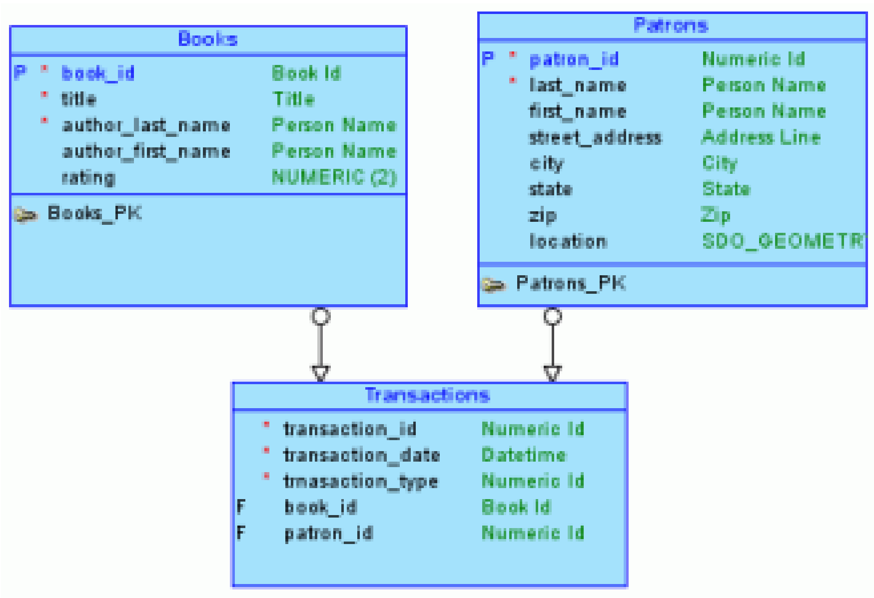
Books and Transactions- one-to-many. Each book can be involved in multiple sequential transactions. Each book can have zero or one active checkout transactions; a book that is checked out cannot be checked out again until after it has been returned.

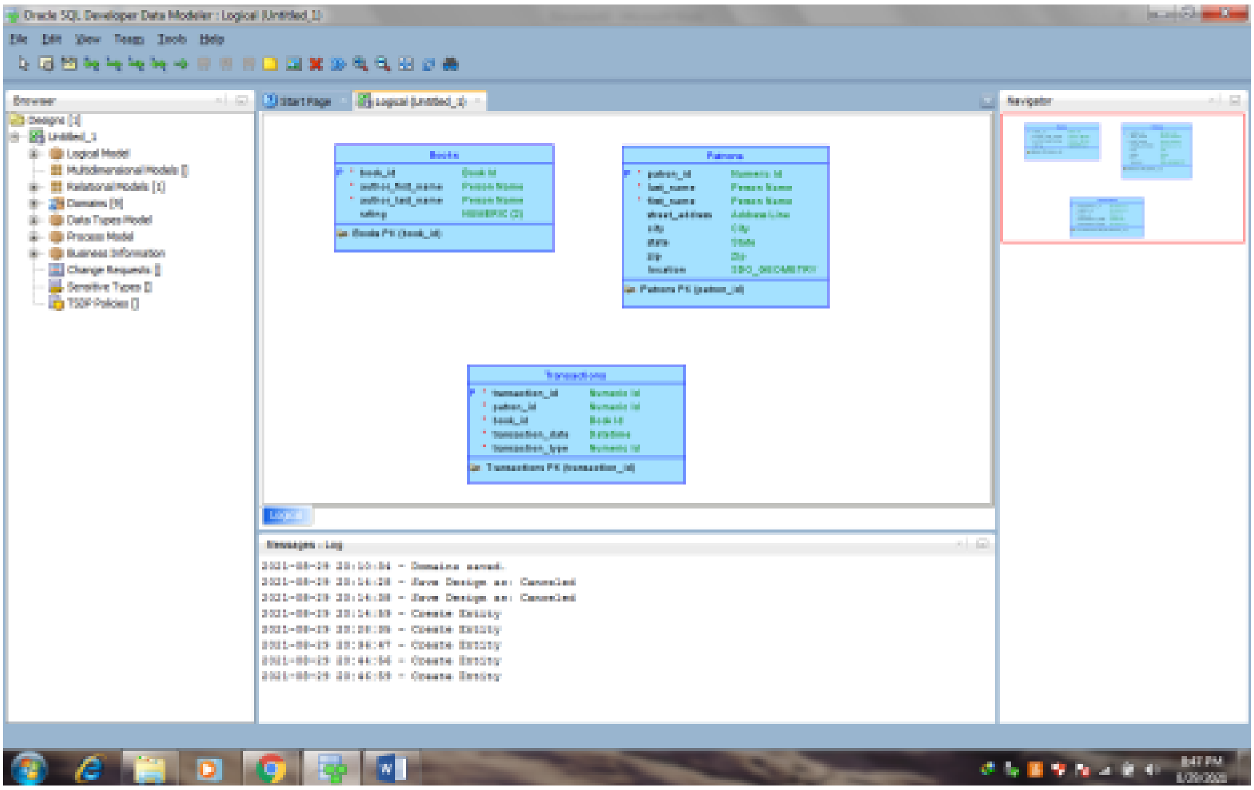
Patrons and Transactions- one-to-many. Each patron can be involved in multiple sequential and simultaneous transactions. Each patron can check out one or many books in a visit to the library, and can have multiple active check out transactions reflecting several visits; each patron can also return checked out books at any time**.**

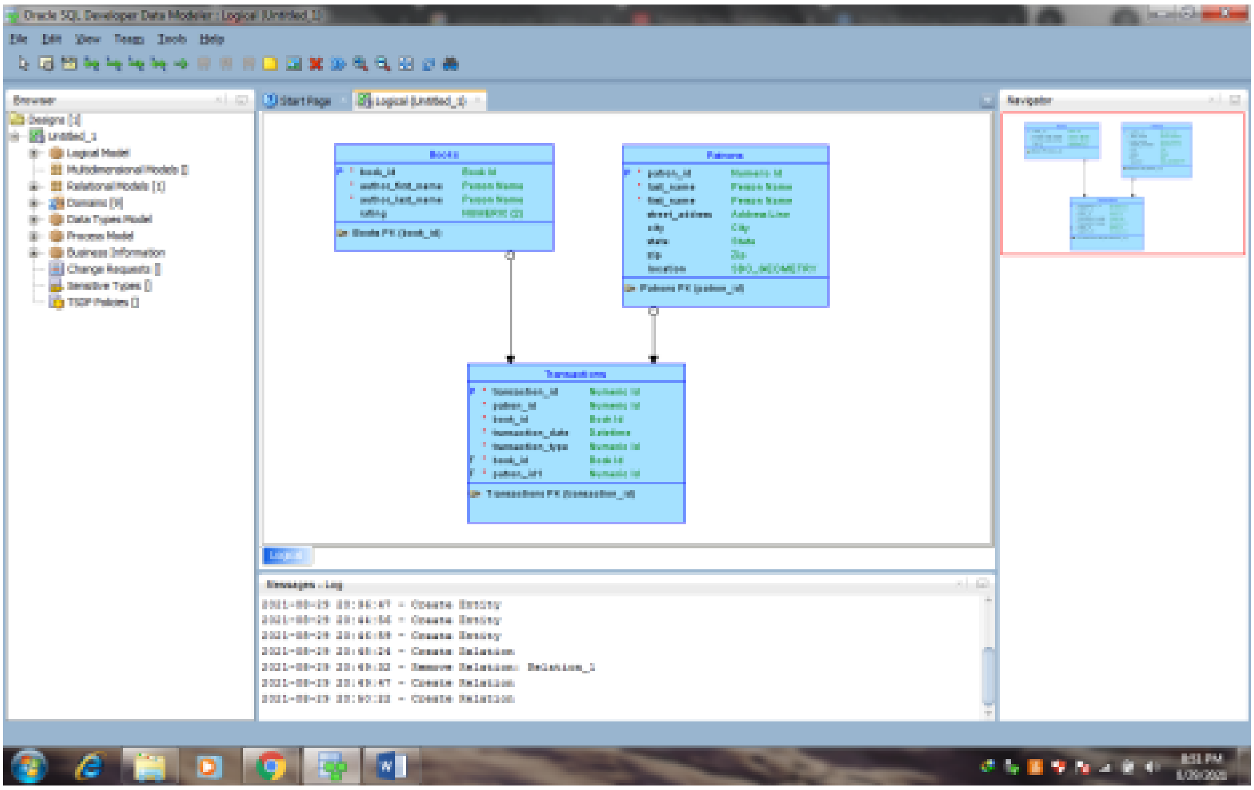
Steps to Create Relationships

1. In the logical model pane in the main area, arrange the entity boxes as follows- Books on the left, Patrons on the right, and Transactions either between Books and Patrons or under them and in the middle. (If the pointer is still cross-hairs, click the Select icon at the top left to change the pointer to an arrow.)

2. Click the New 1- N Relation icon.
3. Click first in the Books box, then in the Transactions box. A line with an arrowhead is drawn from Books to Transactions.
4. Click the New 1- N Relation icon.
5. Click first in the Patrons box, then in the Transactions box. A line with an arrowhead is drawn from Patrons to Transactions.
6. Optionally, double-click a line (or right-click a line and select Properties) and view the Relation Properties information.


Books
P * book_id Book Id
* title Title
* author_last_name Person Name
author_first_name Person Name
rating NUMERIC (2)
Books_PK
Patrons
P * patron_id Numeric Id
* last_name Person Name
first_name Person Name
street_address Address Line
city City
state State
zip Zip
location SDO_GEOMETR
Patrons_PK
Transactions
* transaction_id Numeric Id
* transaction_date Datetime
* trnasaction_type Numeric Id
F book_id Book Id
F patron_id Numeric Id



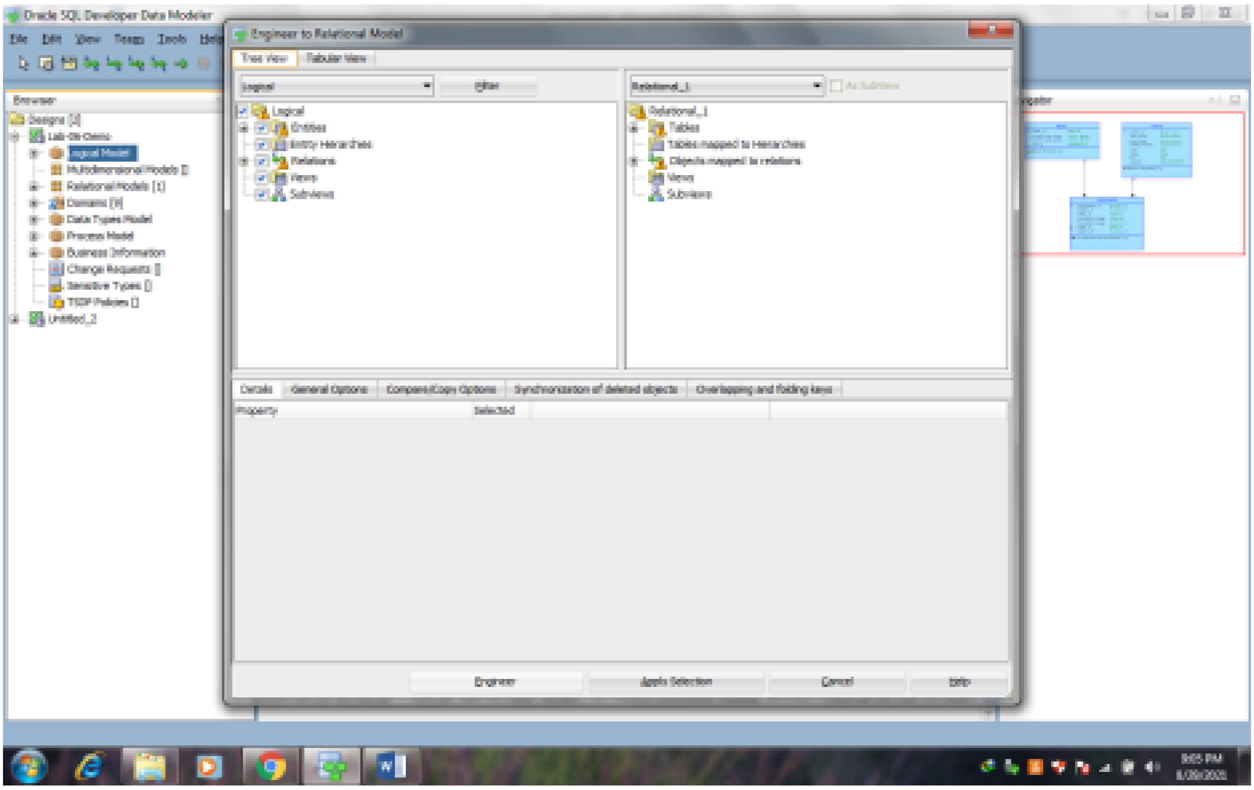


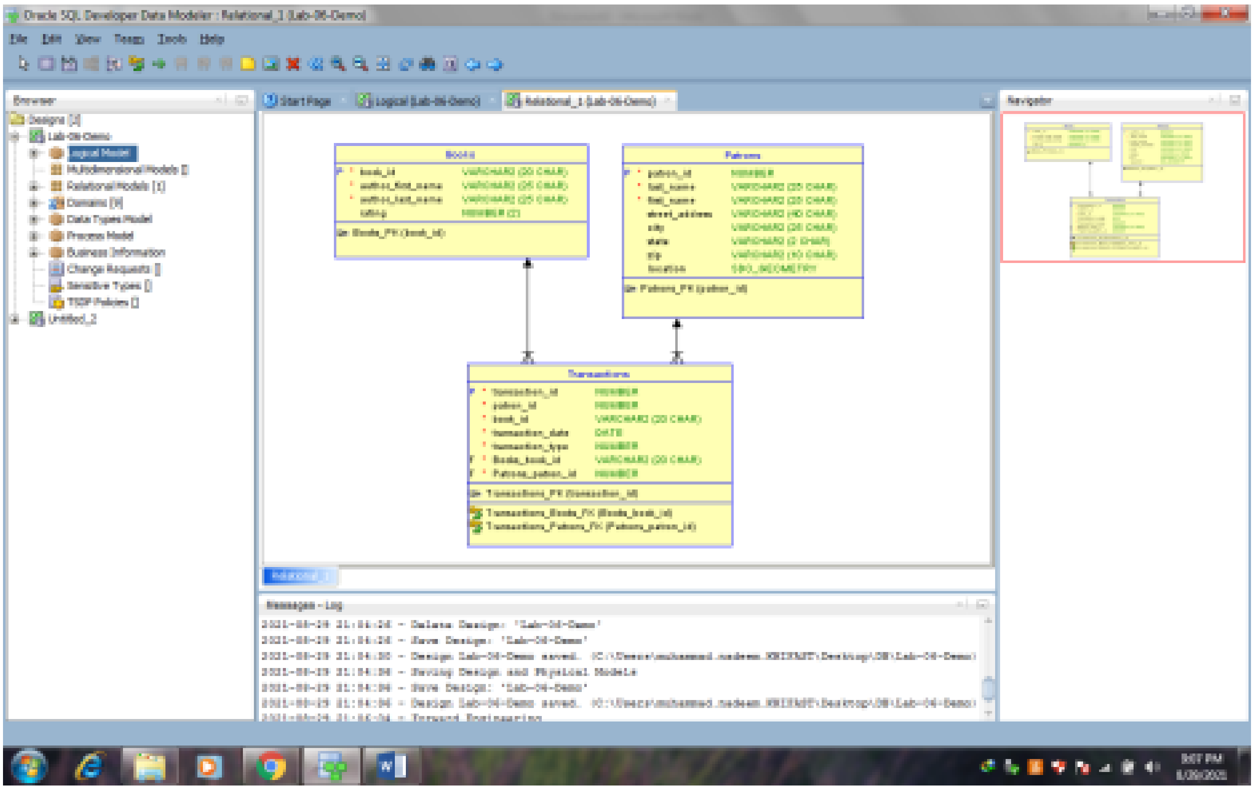
2. Develop the Relational Model

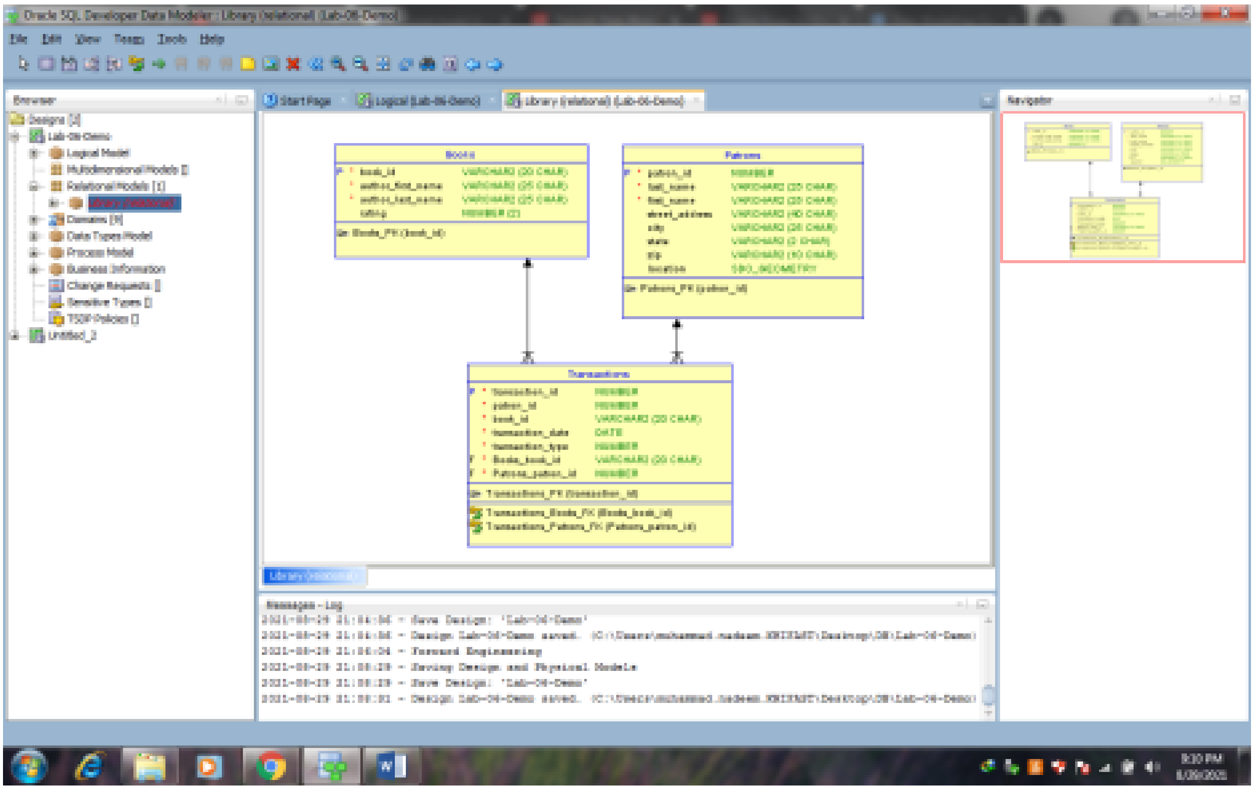
The relational model for the library tutorial data base consists of tables that reflect the entities of the logical model (Books, Patrons, and Transactions) and all attributes of each entity. In the simplified data model for this tutorial, a single relational model reflects the entire logical model; however, for other data models we can create one or more relational models, each reflecting all or a sub set of the logical model. (To have a relational model reflect a subset of the logical model, use the "filter" feature in the dialog box for engineering a relational model.).

Develop the relational model as follows

1. With the logical model selected, click **Design**, then **Engineer to Relational Model**. The Engineering dialog box is displayed.
2. Accept all defaults (do not filter), and click **Engineer**. This causes the Relational_1 model to be populated with tables and other objects that reflect he logical model.
3. Optionally, expand the Relational Models node in the object browser on the left side of the window, and expand Relational_1and nodes under it that contain any entries (such as Tables and Columns), to view the objects created.
4. Change the name of the relational model from Relational_1 to something more meaningful for diagram displays, such as Library(relational). Specifically, right-clickRelational_1 in the hierarchy display, select **Properties**, in the General pane of the Model Properties-<name>(Relational) dialog box specify **Name** as Library(relational), and click **OK**.


Engineer to Relational Model
Engineer
Apply Selection
Cancel
Help


Books
Patrons
Transactions



3. Generate DDL

Develop the physical model as follows

1. Optionally, view the physical model before we generate DDL statements
   a. With the Library logical model selected, click Physical, then Open Physical Model. A dialog box is displayed for selecting the type of database for which to create the physical model.
   b. Specify the type of database (for example, Oracle Database11g), and click OK. In the hierarchy display on the left side of the window, a Physical Models node is added under the Library relational model node, and a physical model reflecting the type of database is created under the Physical Models node.
   c. Expand the Physical Models node under Library (the relational model), and expand the newly created physical model and nodes under it that contain any entries (such as Tables and Columns), to view the objects created.

2. Click File, then Export, then DDL File.
3. Select the database type (for example, Oracle Database11g) and click Generate. The DDL Generation Options dialog box is displayed.
4. Accept all defaults, and click OK. A DDL file editor is displayed, with SQL statements to create the tables and add constraints. (Although we can edit statements in this window, do not edit any statements for this tutorial exercise.)
5. Click Save to save the statements to a SQL script file (for example, create_library_objects.sql) on were local system.
Later, run the script (for example, using a database connection and SQL Worksheet in SQL Developer) to create the objects in the desired database.
6. Close to close the DDL file editor.

4.

Save the Design

Save the design by clicking File, then save. Specify the location and name for the XML file to contain the basic structural information (for example, library_design.xml).

A directory or folder structure will also be created automatically to hold the detailed information about the design. Continue creating and modifying design objects, if we wish. When we are finished, save the design again if we have made any changes, then exit SQL Developer Data Modeler by clicking File, then Exit.

Tasks:

Question 1: A popular online bookstore keeps track of the people who use its website to browse and buy books. Each person can place multiple orders, but an order must always belong to exactly one person. Every order can include several books, and a book can appear in many different orders. Some books are part of special collections, which group multiple titles together. Each book has a unique identifier, a price, and a limited stock quantity that cannot be exceeded when fulfilling orders. The store also records shipping details for each order, ensuring that no two shipments for the same order happen at the same time, and it applies discounts only when certain combinations of books are purchased together.

Question 2: A small city library issues library cards to all members. Each cardholder can borrow multiple books, but a book can only be borrowed by one person at a time. Every borrowing transaction is linked to the library card used, ensuring no one exceeds the borrowing limit of five books. The library also tracks each book's genre and publication year.

Question 3: A fitness center manages its members and the classes they attend. Each member can enroll in multiple fitness classes, and for each class, their attendance is recorded to track participation. Each class session can hold only a limited number of members. Trainers are assigned to classes, and each trainer can lead several classes, but one class has only one trainer.

Question 4: A university department tracks the professors and the courses they teach. Each course is assigned a unique classroom, and no two courses occur in the same classroom at the same time. Professors can teach multiple courses, but each course is taught by only one professor. Each course also has a maximum number of students it can accommodate.

Question 5: An online music platform stores information about users and the playlists they create. Each playlist contains multiple songs, and a song can appear in multiple playlists. Songs belong to specific albums, and albums are tied to artists. Users can only edit playlists they created, and each playlist must have at least one song.

Question 6: A hospital keeps track of patients, doctors, and appointments. Each patient can have multiple appointments, but each appointment is with only one doctor. Doctors can see many patients, but no doctor can have overlapping appointments. Each appointment also includes prescribed medication and dosage details.

Question 7: A car rental service maintains records of customers and the vehicles they rent. Each customer can rent multiple cars over time, but each car can be rented by only one customer at a time. Vehicles belong to specific categories, and the rental rates depend on the vehicle type. Each rental transaction records the start and end date.

Question 8: A photography studio manages photographers, clients, and photoshoots. Each client can book multiple photoshoots, but a photoshoot involves only one client. Photographers can handle several photoshoots, but each photoshoot has only one photographer. Each photoshoot produces multiple edited images stored in the studio's gallery.

Question 9: A university library keeps track of e-books and registered students. Each student can borrow multiple e-books, but each e-book is restricted to one active borrower at a time. E-books belong to certain subjects, and students must return e-books before borrowing new ones. Some e-books have special access restrictions.

Question 10: A cinema maintains records of movies, showtimes, and attendees. Each movie is shown in multiple showtimes, but each showtime occurs in one specific auditorium. Attendees book tickets for showtimes, and no seat can be double-booked. Each showtime has a maximum capacity, and movies are categorized by genre.